

1. Considerate la grammatica di cui sono date le produzioni, e dove si assume che il simbolo di start sia S : $S \rightarrow aSA$, $S \rightarrow SAb$, $S \rightarrow aAb$, $S \rightarrow bAa$, $A \rightarrow bAb$, $A \rightarrow aAa$, $A \rightarrow \epsilon$, trovate una derivazione per la parola $aaabbbbaabb$, e per ogni passo indicate quale produzione applicate.
2. Eseguite la SECD

$([], \perp, \mathbf{apply}(\mathbf{fun}(f, x, \mathbf{apply}(x, \mathbf{apply}(\mathbf{fun}(g, y, \mathbf{times}(y, 2)), 1))), \mathbf{fun}(h, z, \mathbf{sum}(5, z))) :: [] :: []$

dove eseguire significa che dovete fare tutti i passaggi per arrivare al risultato usando le regole per la SECD che trovate nei fogli allegati. Per risultato si intende una SECD in cui non potete applicare più alcuna regola, non è detto che la SECD termini in uno stato finale.

3. Considerate il seguente pezzo di pseudo-codice di un linguaggio *imperativo* (quindi non corrisponde ad un preciso linguaggio di programmazione). I parametri, quando non indicato diversamente in una domanda, sono passati per *valore*. Ogni linea di codice è identificata da un numero, le parentesi graffe indicano i blocchi ed il '*' indica la moltiplicazione tra interi. Le funzioni restituiscono un valore, che può anche essere una funzione, ed il valore restituito è indicato dalla keyword **return**. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra, ad esempio se considerate $f(x) + g(y)$ può accadere che i side effects dell'esecuzione della $f(x)$ si ripercuotano sull'esecuzione della $g(y)$ che avviene *dopo* l'esecuzione della $f(x)$.

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   int f (int y) {
05     y = x + z;
06     return y};
07   int m (int x) {;
08     return y + x};
09   int h1 (int w) {
10     int x = 3;
11     x = f(x) * w + x;
12     return x};
13   int h2 (int x) {
14     int w = 1;
15     w = f(w) + w;
16     return w};
17   int g (int n) {
18     int x = 2;
19     int z = 3;
20     x = h1(x);
21     z = h2(z);
22     y = m(x) + f(n) + n;
23     return y};
24 {int x = 1;
25   int z = 1;
26   x = g(x) + h2(y) + y;}}
```

- (a) che valore contiene la variabile x alla linea 26 se lo scope è statico
 - (b) che valore contiene la variabile x alla linea 26 se lo scope è statico e i parametri delle funzioni sono tutti passati per riferimento
 - (c) che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella dello shallow binding
 - (d) che valore contiene la variabile x alla linea 26 se lo scope è dinamico, la politica è quella dello shallow binding e i parametri delle funzioni sono tutti passati per riferimento
 - (e) che valore contiene la variabile x alla linea 26 se lo scope è dinamico e la politica è quella del deep binding
4. Considerate il seguente tipo:

$(\mathbf{int} \rightarrow \mathbf{bool}) \rightarrow (\mathbf{int} \rightarrow \mathbf{bool}) * \mathbf{int} \rightarrow \mathbf{int}$

scrivete un termine in Ocaml-like che ha tale tipo ed inferitene il tipo.